

Practical: 5

Implementation of a knapsack problem using dynamic programming.

25/08/2026

Aim: To Implement the knapsack problem using dynamic programming.

Algorithm:

Algorithm:-

Input: n = number of items, w = max knapsack capacity
 $w[i]$ = weight of item i
 $v[i]$ = value of item i

Output: Max possible values
 Selected and Non-selected items

Step 1: Start
 Step 2: Read the No. of item n
 Step 3: Read Max Capacity of the knapsack w
 Step 4: for each item $i = 1$ to n .
 i) $w[i]$ and
 ii) $v[i]$.

Step 5: Create a DP table $dp[i][j]$, where:
 i) i represent the first i items
 ii) j current knapsack capacity
 iii) $dp[i][j]$ Max value using i with Capacity j .

Step 6: for $i = 0$ to n do,
 $dp[i][0] = 0$;
 for $j = 0$ to w , do
 $dp[0][j] = 0$

Step 7: Fill the DP table
 for $i = 1$ to n
 for $j = 1$ to w
 if $(w[i] \leq j)$
 $v[i] + dp[i-1][j-w[i]]$
 $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$

Step 8: $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$
 Step 9: if $(w[i] > j)$
 $dp[i][j] = dp[i-1][j]$; $dp[n][w]$

Step 10: Let $j = w$ from $i = n$ and move down to 1.
 Step 11: if $dp[i][j] \neq dp[i-1][j]$
 $j = j - w[i]$
 Step 12: Stop.

Program:

```
#include<iostream>
using namespace std;

class Knapsack {

private:
    int w[10];    // weights
```

```
int v[10];    // values
int n;        // number of items
int W;        // capacity
int dp[10][100]; // dp table
```

public:

```
// ?? Get Input ??
void getInput() {
    cout << "Enter number of items : ";
    cin >> n;
    cout << "Enter knapsack capacity : ";
    cin >> W;
    for(int i = 1; i <= n; i++) {
        cout << "Item " << i << " weight and value : ";
        cin >> w[i] >> v[i];
    }
}
```

```
// ?? Build dp table ??
void solve() {

    // base case - row 0 and col 0
    for(int i = 0; i <= n; i++) dp[i][0] = 0;
    for(int j = 0; j <= W; j++) dp[0][j] = 0;
```

```
    // fill table
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= W; j++) {

            if(w[i] <= j) {
                // take or not take
                dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]],
                    dp[i-1][j]
                );
            } else {
                // cannot take item i
                dp[i][j] = dp[i-1][j];
            }
        }
    }
}
```

```
// ?? Print dp table ??
void printTable() {
    cout << "\nDP Table:" << endl;
```

```
cout << " ";
for(int j = 0; j <= W; j++)
    cout << j << " ";
cout << endl;

for(int i = 0; i <= n; i++) {
    cout << "i=" << i << " ";
    for(int j = 0; j <= W; j++)
        cout << dp[i][j] << " ";
    cout << endl;
}
}

// ?? Find selected items ??
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " << endl;
        }
    }
}

// ?? Show result ??
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
}

};

// ?? MAIN ??
int main() {

    Knapsack k;

    k.getInput();
    k.solve();
}
```

```
k.printTable();
k.showResult();
k.findItems();

return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D x + v
Enter number of items : 2
Enter knapsack capacity : 4
Item 1 weight and value : 2 4
Item 2 weight and value : 6 4

DP Table:
  0 1 2 3 4
i=0 0 0 0 0 0
i=1 0 0 4 4 4
i=2 0 0 4 4 4

Maximum Value = 4

Selected Items:
Item 2 (w=6, v=4) NOT taken
Item 1 (w=2, v=4) TAKEN
```

Time Complexity:

Time Complexity:

```
for (int i = 1; i <= n; i++) {  $\rightarrow T_1(n) = O(n)$ 
  for (int j = 1; j <= w; j++) {  $\rightarrow T_2(w) = O(w)$ 
    if (w[i][j] == 0)  $\rightarrow O(1)$ 
      dp[i][j] = max(v[i] + dp[i-1][j-w[i]],
        dp[i-1][j]);  $\rightarrow O(1)$ 
    }
  }
}
```

Nested loop $n \times w = T(n, w) = O(n, w)$

Hence The Total dp calculation

$n \times w = O(1)$

$T(n, w) = O(n, w)$

$T(n, w) = O(n, w)$

Best = Average = Worst = $O(n, w)$

Conclusion:

The Implementation of the knapsack problem using dynamic programming successfully executed.

And the time complexity analysis is same for all the three cases Best case = Average case = Worst

Case = $O(n, w)$

Where n is Number of items and w is the Maximum capacity of the knapsack.